Database Architecture Specification AI-Powered Hospital Management System

Enterprise-Grade Document & Schema Design for MedAI OS

System Architecture:	React (Frontend) + Spring Boot (Microservices) + MongoDB (Data Layer)
Author:	Senior Software Architect
Date:	June 12, 2026
Document Version:	v1.0.0
Classification:	Confidential / Internal Engineering Only

1. Executive Summary & Architectural Design

This document details the enterprise database schema for the **AI-Powered Hospital Management System (MedAI OS)**. Leveraging a modern full-stack tech container consisting of a **React** single-page application, a **Spring Boot** enterprise backend framework, and a distributed **MongoDB** document database, this schema balances structural operational integrity with the data fluidity required by modern Artificial Intelligence workloads (e.g., LLM medical summarization, structured predictive diagnostics, and real-time streams).

1.1 MongoDB Modeling Philosophy

Unlike traditional Relational Database Management Systems (RDBMS) that rely heavily on normalized multi-table joins, this design is structured specifically around MongoDB's native paradigms:

- **Controlled Embedding vs. Referencing:** High-frequency, atomic, and bounded sub-datasets (e.g., items within an invoice, logs within a notification, metadata arrays) are deeply embedded as sub-documents to fulfill reads within a single network turn. Unbounded datasets or high-concurrency entities (e.g., patient clinical records, pharmacy inventory, structural user logs) use document references (***ObjectId*** strings).
- **Polymorphism:** Accommodates varying schemas for multiple distinct electronic medical events (such as prescriptions, diagnostics, and triage) inside singular collections, lowering application-layer mapping overhead.
- **Write Performance Optimization:** Minimizes multi-document transactional overhead while implementing strategic secondary indexes to allow high-throughput querying across complex, cross-referenced domains.

1.2 Spring Boot & Security Integrations

The backend layer leverages **Spring Data MongoDB** for object-relational mapping (ODM), utilizing annotations like `@Document`, `@Id`, `@DBRef` (sparingly), and `@Indexed`. Security is handled natively via Spring Security and JWT, utilizing cryptographic hashes within the `users` collection. All clinical queries are designed to be bounded by tenancy and patient identifiers to enforce tight HIPAA compliance.

2. Conceptual Entity-Relationship Description

Since MongoDB uses a flexible, document-oriented data model, structural relationships are established via logical foreign keys or sub-document nesting rather than physical database-level constraints. Below is the behavioral system-wide topology:

- **One-to-Many Nesting (Embedded):** `Invoices` completely encapsulates an array of `LineItems` sub-documents. `Appointments` encapsulates `StatusHistory` blocks. `EMRRecords` encapsulates `Vitals`, `Symptoms`, and `AIEvaluation` blocks.
- **One-to-Many Linking (Referencing):** A single `User` (Doctor) maps to many `Appointments` and `Consultations` via the presence of a `doctorId` field inside the destination targets. A `Patient` is linked directly to `Admissions`, `EMRRecords`, and `BillingInvoices` via a `patientId` reference.

- **Many-to-Many Resolution:** Accomplished elegantly by injecting arrays of references. For example, a `LabOrder` references a single `Patient` and a single `Doctor`, but maps an array of `testIds` pointing to a catalog collection of static `LabTests`.

3. Detailed Module Collection Specifications

The following sections outline every document collection across the 12 core functional modules. Each definition explicitly provides field names, BSON types, indexing configurations, and logical relationships.

Module 1: Authentication & Authorization

Manages system-wide identity access management (IAM), Role-Based Access Control (RBAC), and authentication histories.

Collection: users

Field	BSON Type	Key Type	Description / Relationships
_id	ObjectId	PK	Unique surrogate system identifier for the user.
username	String	`Unique`	Unique employee identity or system login handle.
passwordHash	String	-	Bcrypt/Argon2 securely salted password hash string.
email	String	`Unique`	Primary communication email. Index for recovery.
roles	Array<String>	-	RBAC roles assigned (e.g., `["DOCTOR", "ADMIN"]`).
permissions	Array<String>	-	Fine-grained permission nodes (e.g., `["emr:read", "billing:write"]`).
isActive	Boolean	-	Flag denoting account accessibility status.
metadata	Document	-	Embedded document containing profile details (name, phone, specialty).

Collection: auth_audit_logs

Field	BSON Type	Key Type	Description / Relationships
_id	ObjectId	PK	Unique record tracking identifier.
userId	ObjectId	FK	References `users._id`. Maps ownership of log event.
loginTime	Date	`Index`	Timestamp of authorization execution. TTL index targets here.
ipAddress	String	-	IPv4/IPv6 client machine network identifier.
status	String	-	Outcome states: `SUCCESS`, `FAILED`, `LOCKED`.

Module 2: Patient Management

Maintains demographic, structural identity details, and base configurations for individuals receiving care.

Collection: patients

Field	BSON Type	Key Type	Description / Relationships
_id	ObjectId	PK	Unique master patient sequence token.
medicalRecordNumber	String	Unique	Human-readable unique alpha-numeric MRN identifier.
firstName	String	-	Patient legal given name. Secondary indexed with lastName.
lastName	String	-	Patient legal family surname.
dateOfBirth	Date	-	ISO standard date of birth for age parsing.
gender	String	-	Biological sex registration classification.
emergencyContact	Document	-	Embedded document: `{ name, relationship, phone }`.
insuranceDetails	Array<Doc>	-	Embedded array containing active insurance policies and policy IDs.

Module 3: Appointment Management

Tracks scheduled touchpoints between clinical practitioners and patient groups.

Collection: appointments

Field	BSON Type	Key Type	Description / Relationships
_id	ObjectId	PK	Unique primary reference for the scheduling event.
patientId	ObjectId	FK	References `patients._id`. Compound indexed with scheduleTime.
doctorId	ObjectId	FK	References `users._id` representing clinical practitioner.
scheduleTime	Date	Index	Explicit start timestamp for slot booking allocation.
durationMinutes	Int32	-	Duration scope of reservation blocks. Default 30.
status	String	-	Enums: `SCHEDULED`, `CHECKED_IN`, `COMPLETED`, `CANCELLED`.
type	String	-	Modality: `IN_PERSON`, `TELEHEALTH`, `AI_SCREENING`.
statusHistory	Array<Doc>	-	Embedded state tracking: `[{ status, updatedBy, timestamp }]`.

Module 4: Doctor Consultation

Captures dynamic runtime workflows when a clinician evaluates a patient encounter directly.

Collection: consultations

Field	BSON Type	Key Type	Description / Relationships
_id	ObjectId	PK	Unique consultation lifecycle identifier.
appointmentId	ObjectId	FK	References source `appointments._id`. (Can be null if walk-in).
patientId	ObjectId	FK	References `patients._id`. Master owner tracking.
doctorId	ObjectId	FK	References `users._id` representing clinical processor.
chiefComplaint	String	-	Primary narrative stating motivation for consultation request.
clinicalNotes	String	-	Raw text block input compiled by human doctor during interface active state.
aiCoPilotSuggestions	Document	-	Embedded dynamic results structure from active real-time AI context parsing.

Module 5: Electronic Medical Records (EMR)

Core health records matrix capturing multi-dimensional diagnostic metrics, history, and heavy AI inference payloads.

Collection: emr_records

Field	BSON Type	Key Type	Description / Relationships
_id	ObjectId	PK	Unique health-record timeline block hash tracking token.
patientId	ObjectId	FK	References `patients._id`. Heavy multi-index querying anchor.
consultationId	ObjectId	FK	References `consultations._id` linking transactional generation trace.
vitals	Document	-	Embedded: `{ bpSystolic, bpDiastolic, heartRate, tempCelsius, spo2 }`.
diagnoses	Array<Doc>	-	List of entries: `[{ icd10Code, description, type: PRIMARY/SECONDARY }]`.
aiInsights	Document	-	Embedded document housing heavy analytical payloads (details below).
createdAt	Date	Index	Chronological record sorting timestamp.

Deep Dive: Embedded AI Insights Document Structure

To prevent costly relational lookups across large sets of clinical text, the `aiInsights` field embeds structural engine predictions directly inside the EMR record:

```
aiInsights: {
  llmSummary: "Patient presents with escalating structural hypertension secondary to chronic renal patterns.",
  riskScores: { cardiovascular10Yr: 0.142, sepsisRiskIndex: 0.02 },
  anomalyDetected: true,
  recommendedProtocols: ["ICD10-I10-E", "ORDER_RENAL_PANEL"],
  vectorEmbedding: [-0.0124, 0.4561, 0.9112, ... (1536 dimensions for similarity search)]
}
```

Module 6: Laboratory Management

Manages execution requests and result returns for body fluid, tissue, and analytical diagnostic requests.

Collection: lab_orders

Field	BSON Type	Key Type	Description / Relationships
_id	ObjectId	PK	Unique laboratory transactional identifier tracking code.
patientId	ObjectId	FK	References `patients._id`. Identifying record subject.
orderedBy	ObjectId	FK	References `users._id` representing originating physician.
testCodes	Array<String>	-	Array of target procedures being requested (e.g., `["CBC", "CMP"]`).
specimenDetails	Document	-	Embedded tracking detail: `{ type: "BLOOD", barcode: "L309112" }`.
results	Array<Doc>	-	Embedded test returns: `[{ testCode, parameter, value, referenceRange, status }]`.
aiAnomalyFlag	Boolean	-	Flag marked true if value structures fall out-of-bounds via pipeline parsing.
status	String	-	Workflow states: `ORDERED`, `SAMPLE_COLLECTED`, `PROCESSING`, `COMPLETED`.

Module 7: Pharmacy Management

Tracks active medication inventory catalogs and the structured verification and dispensing of therapeutic orders.

Collection: medication_inventory

Field	BSON Type	Key Type	Description / Relationships
_id	ObjectId	PK	Unique SKU database identifier.
ndcCode	String	Unique	National Drug Code index unique identifier.
name	String	-	Commercial/Generic pharmacological naming label.
stockLevel	Int32	-	Current atomized unit quantity available within standard storage units.
reorderThreshold	Int32	-	Safety threshold minimum value. Triggers procurement pipelines.
batchDetails	Array<Doc>	-	Embedded safety mapping structure: `[{ batchNumber, expiryDate, quantity }]`.

Collection: prescriptions

Field	BSON Type	Key Type	Description / Relationships
_id	ObjectId	PK	Unique prescription tracking hash token.
patientId	ObjectId	FK	References `patients._id` identifying targeted receiver.
doctorId	ObjectId	FK	References `users._id` mapping authorizing clinician.
medications	Array<Doc>	-	Embedded list: `[{ medicationId, genericName, dosage, frequency, durationDays }]`.
aiInteractionCheck	Document	-	Embedded validation array: `{ contraindicationsDetected: false, alertLevel: "NONE" }`.
status	String	-	Dispensation timeline states: `ISSUED`, `FILLED`, `DISPENSED`.

Module 8: Billing & Payments

Financial ledger management tracking service costs, invoice generation, allocations, and claims processing.

Collection: invoices

Field	BSON Type	Key Type	Description / Relationships
_id	ObjectId	PK	Unique internal primary system invoice ID key.
invoiceNumber	String	Unique	Human-facing audit tracker format identifier code.
patientId	ObjectId	FK	References `patients._id` designating single billable entity.
lineItems	Array<Doc>	-	Embedded elements: `[{ serviceName, quantity, unitPrice, insuranceCovered, netAmount }]`.
totals	Document	-	Embedded pricing summaries: `{ grossAmount, totalInsurancePaid, patientCopayDue }`.
paymentStatus	String	-	Enums state control: `UNPAID`, `PARTIALLY_PAID`, `SETTLED`, `DISPUTED`.

Module 9: Admission Management

Governs lifecycle logistics of Inpatient Department (IPD) stays, bed telemetry tracking, and transfers.

Collection: admissions

Field	BSON Type	Key Type	Description / Relationships
_id	ObjectId	PK	Unique admission transactional reference object.
patientId	ObjectId	FK	References `patients._id` linking admitted individual.
admissionType	String	-	Routing categories: `ELECTIVE`, `TRANSFER`, `EMERGENCY`.
locationAllocation	Document	-	Embedded dynamic positional data: `{ wing: "NORTH_ICU", room: "402A", bedNumber: 3 }`.
admitDate	Date	-	Timestamp tracking check-in execution.
dischargeDetails	Document	-	Nullable embedded document initialized upon resolution: `{ dischargeDate, condition, summary }`.

Module 10: Emergency Management

High-concurrency triage tracking mechanism optimized for swift ingestion, routing, and mobile trauma processing.

Collection: emergency_encounters

Field	BSON Type	Key Type	Description / Relationships
_id	ObjectId	PK	Unique Emergency Room operational sequence token.
patientId	ObjectId	FK	References `patients._id`. (Can be null for anonymous unidentified trauma cases).
triageLevel	Int32	Index	ESI standard score representation scale: 1 (Immediate Resuscitation) to 5 (Non-Urgent).
presentingSymptoms	Array<String>	-	Active presentation symptoms array declared at encounter point.
aiPriorityScore	Decimal128	-	Real-time machine prioritization routing ranking value based on vitals array.
assignedTraumaTeam	Document	-	Embedded responsive reference set: `{ leadPhysicianId, codeType, activeStatus }`.

Module 11: Notifications

Funnels event streams into alert matrices via multi-channel pathways (Web Sockets, SMS, Push Notification protocols).

Collection: notifications

Field	BSON Type	Key Type	Description / Relationships
_id	ObjectId	PK	Unique transient message notification object tracking key.
recipientUserId	ObjectId	FK	References `users._id` or `patients._id` target receiver. Structured index.
channel	String	-	Operational mechanism: `IN_APP_PUSH`, `SMS`, `EMAIL`, `PAGER`.
priority	String	-	Escalation parameters: `CRITICAL_CODE`, `HIGH`, `ROUTINE`.
payload	Document	-	Flexible embedded content: `{ title, body, actionableDeeplinkRoute }`.
isRead	Boolean	-	Delivery response tracking verification state flag indicator.
timestamp	Date	Index	Creation instant time tracking block marker. Backed by 30-day TTL expiration index.

Module 12: Reports & Analytics

Aggregates pre-computed historical health data metrics, resource utilization curves, and AI analytical projections.

Collection: analytical_snapshots

Field	BSON Type	Key Type	Description / Relationships
_id	ObjectId	PK	Unique analytic extraction record sequence hash key identifier.
metricDomain	String	`Index`	Context scope index targets: `BED_OCCUPANCY`, `REVENUE_PER_DEPT`, `AI_DIAG_ACCURACY`.
snapshotPeriod	String	-	Time tracking encapsulation range: `DAILY_2026_W24`, `MONTHLY_2026_M06`.
aggregatedMetrics	Document	-	Deeply structural nested document holding compiled numerical summary statistics array.
predictiveForecasts	Document	-	Embedded trends structured from external machine learning processing nodes running evaluations.

4. Database Optimization Strategy & Indexing Manifest

To ensure sub-millisecond query responses across massive relational structures without traditional joins, specific single-field, compound, and specialized indexes must be programmatically generated via Spring Boot lifecycle configurations or raw migrations.

Critical Enterprise Index Configuration

Target Collection	Index Keys Defined	Index Type	Architectural Motivation
users	`{ username: 1 }`, `{ email: 1 }`	Unique Standard	Enforces credential isolation during auth handshakes.
patients	`{ medicalRecordNumber: 1 }`	Unique Standard	Accelerates master clinical profile retrieval.
appointments	`{ doctorId: 1, scheduleTime: 1 }`	Compound	Prevents double-booking via rapid calendar queries.
emr_records	`{ patientId: 1, createdAt: -1 }`	Compound Desc	Pulls patient health history timelines sequentially.
emr_records	`{ "aiInsights.vectorEmbedding": "2dsphere" }`	Vector / Special	Powers semantic cognitive lookup of similar clinical profiles.
emergency_encounters	`{ triageLevel: 1, aiPriorityScore: -1 }`	Compound	Maintains real-time priority queues for emergency triage boards.
notifications	`{ timestamp: 1 }`	TTL Index	Configured with `expireAfterSeconds: 2592000` to auto-purge logs after 30 days.

5. Full-Stack Integration Flow (React + Spring Boot + MongoDB)

The end-to-end integration operates smoothly across layers to maximize operational speed while keeping data consistency tight:

1. **Frontend State Capture (React):** Form interfaces manage modular inputs (e.g., admitting an ER trauma patient) and structure them as single, deeply cohesive JSON models, sending them downstream via secure REST endpoints backed by Axios/TanStack Query.
2. **Enterprise Routing Control (Spring Boot):** The API layer ingests payloads into Java records/DTOs. The service layer invokes explicit validation pipelines, executes synchronous context comparisons (such as checking drug-to-drug interactions using an embedded utility block), and hands models off to Spring Data repositories.
3. **Persistence and Aggregation Layer (MongoDB):** Documents are committed atomically within high-performance write paths. For complex analytics, rather than relying on heavy real-time lookups, MongoDB's **Aggregation Pipeline** utilizes structural filtering stages (***$match, $group, $project***) to stream optimized analytical snapshots cleanly into dashboard interfaces.